CSC262 · ARTIFICIAL INTELLIGENCE – I

LAB PROJECT PROPOSAL · FALL 2026

CyberTrail: AI-Assisted Cybersecurity Evidence Investigation Search System

PROJECT CATEGORY

Cybersecurity Decision Support · Artificial Intelligence Search

SUBMITTED BY

Khawajah Mubashir — SP25-BCT-019
Muhammad Zeeshan Iqbal — SP25-BCT-037
Uzair Ahmad — SP25-BCT-051

SUBMITTED TO

Ma’am Farah Saeed

Department of Computer Science
COMSATS University Islamabad · Islamabad Campus

26 OCTOBER 2026

Table of Contents

Abstract · 1 Introduction · 1 Proposed System · 2 Benefits · 2 Scope · 2 Modules · 3 Limitations · 3 Tools and Technologies · 3 Conclusion · 4 References · 4

Abstract

Security analysts often investigate alerts by following relationships among users, endpoints, processes, servers, and network events. CyberTrail is an educational decision-support application that represents a small synthetic set of these relationships as a directed graph. Starting from an alert node, the user selects a target asset and compares BFS, DFS, Uniform-Cost Search, Greedy Best-First, A*, and Hill Climbing for exploring evidence paths. The application visualizes expanded nodes and the selected path and reports link count, estimated analyst review effort, expanded nodes, runtime, and search guarantees. A* uses an admissible graph-distance heuristic for the defined positive edge costs. The project demonstrates AI course concepts in problem formulation, graph search, heuristics, informed search, and local-search limitations. Its data and review costs are illustrative; it is not a live threat detector, does not infer attack attribution, and never takes response actions.

1. Introduction

Security operations teams receive alerts from identity, endpoint, server, and network monitoring tools. One alert can be related to multiple accounts, processes, assets, or communications. Analysts therefore need to inspect relationships and decide which evidence to verify. A graph is a useful representation because entities can be represented as nodes and observed relationships as edges. Searching such a graph can help an analyst trace a possible route from an initial alert toward an important asset. However, graph paths can have different numbers of links and different investigation effort costs. A path with fewer links is not necessarily the path with the lowest total review effort. Search algorithms provide systematic ways to explore possible paths and select a solution according to a defined objective. Uninformed algorithms do not use an estimate of distance to the goal. Informed algorithms use a heuristic to guide exploration, while local search keeps limited information and can become trapped. The CSC262 lectures introduce problem formulation, state spaces, BFS, DFS, uniform-cost search, Greedy Best-First, A*, heuristic admissibility, and Hill Climbing. CyberTrail applies these topics to a synthetic cybersecurity evidence-investigation scenario.

The software addresses the educational problem of making alert-evidence relationships easier to explore and compare. A user selects an alert/evidence node as the initial state and a critical asset as the goal. The system follows directed, synthetic telemetry relationships and sums illustrative review-effort costs along candidate paths. It compares six search strategies on the same graph and highlights the difference between fewest links and lowest estimated review effort. The interface animates explored nodes so the search process is visible rather than a hidden result. This project is expected to develop skills in Python, graph representation, state-space search, priority queues, path reconstruction, heuristic design, and API development. It will help students explain the effects of queues, stacks, accumulated path costs, and heuristic estimates. Testing will reinforce how to validate reachable and unreachable goals and compare search results. Students can analyze why A* and UCS agree on least cost for the defined graph while BFS and Greedy can choose a more expensive route. The demonstration also shows how a local-search strategy can follow a misleading lead. The application does not claim that the least-effort path is the true attacker path. The analyst must inspect evidence and use organizational procedures before taking any action.

2. Proposed System

CyberTrail is a local Flask application with a browser-based security-operations interface. It loads a synthetic directed evidence graph from a local JSON file. Graph nodes represent example alerts, users, endpoints, processes, network events, servers, and critical assets. Directed edges represent illustrative telemetry relationships such as a login, process spawn, or network connection. The start node is the selected alert or evidence source, and the target node is an asset chosen by the user. Traversing an edge represents reviewing the related evidence, and its positive cost is an estimated analyst review effort. Edge confidence is displayed as synthetic context but does not affect the path cost and is not interpreted as a probability of compromise. BFS explores by link depth, while DFS uses a stack to explore a branch deeply. UCS orders its frontier by total cost accumulated so far. Greedy Best-First uses an estimate of remaining graph hops, whereas A *combines the accumulated cost with a lower-bound estimate. The A* heuristic is reverse-graph minimum hop distance multiplied by the graph's minimum edge cost. Because all edge costs are positive and at least the minimum edge cost, this heuristic does not overestimate the remaining path cost. Hill Climbing uses a simple graph-layout proximity estimate and moves only to a locally closer adjacent node, making its limitations visible. Users can run one strategy or compare all six strategies on the same source and target. The interface animates expanded nodes and highlights the chosen path. A results table reports outcome, number of evidence links, estimated review cost, expanded nodes, runtime, and guarantees. Input validation rejects invalid scenarios, unknown nodes, or unsupported algorithms. All data is synthetic, and the system provides no live monitoring, threat attribution, or automated response.

3. Advantages/Benefits of Proposed System

1. Relevant cybersecurity use case: models analyst exploration of alert-related evidence.
2. Direct course alignment: demonstrates search formulation, BFS, DFS, UCS, Greedy, A*, heuristics, and Hill Climbing.
3. Visual reasoning: shows explored nodes and the route selected by a search method.
4. Cost-aware comparison: differentiates fewest links from least estimated review effort.
5. Explainable heuristic: documents the A* estimate and why it is admissible for the graph.
6. Limitations are visible: demonstrates that Greedy and Hill Climbing can be misled or non-optimal.
7. Safe and reproducible: uses a local synthetic graph with no real accounts, telemetry, or response actions.

4. Scope

The project covers a local browser interface for searching a synthetic cybersecurity evidence graph. It includes a small scenario with alert, identity, endpoint, process, network, server, and critical-asset nodes. The user can choose a source alert and target asset from the graph. Directed edges model illustrative telemetry relationships. Each edge has a positive, synthetic review-effort cost and a contextual confidence value. Search cost is the sum of review-effort costs; confidence is not treated as a probability and is not part of the search score. The project implements BFS, DFS, UCS, Greedy Best-First, A*, and Hill Climbing. It uses a graph-hop lower bound for the A** heuristic. The interface animates expanded nodes and highlights the selected route. A comparison view reports link count, effort cost, expanded nodes, runtime, and algorithm guarantee. Users can click nodes to inspect details and set the source or target. The API validates scenario, node, and algorithm inputs. Automated tests cover path correctness, differing BFS and UCS outcomes, A*/UCS cost agreement, local-search failure, and unreachable goals. The system does not ingest real logs, endpoint telemetry, or threat-intelligence feeds. It does not detect malware, infer attacker identity, or establish that a path is an actual attack chain. It does not execute containment, block traffic, or modify accounts. Authentication, a production database, concurrent analyst support, and deployment hardening are outside the project scope. Real-world use would require validated data, calibrated effort costs, security controls, and extensive analyst-led testing.

5. Modules

The application is divided into four modules: Synthetic Evidence Graph, Search Algorithm Engine, Graph Visualization and Comparison, and Flask API/Input Validation. Special features include selectable source and target nodes, animated exploration, edge-cost and contextual-confidence tooltips, and comparison of six course search strategies. The main AI module is described below.

5.1 Module 1: Search Algorithm Engine

This module receives the selected graph, start node, goal node, and search algorithm. It represents possible investigation steps as directed transitions between evidence nodes. BFS uses a FIFO queue, while DFS uses a LIFO stack. UCS uses a priority queue ordered by accumulated edge cost; Greedy orders by estimated remaining hops; A *orders by* g(n)+h(n)*. The A* heuristic uses the minimum number of remaining graph links multiplied by the minimum edge cost, which is an admissible lower bound for this graph. Hill Climbing uses a local proximity estimate and can be diverted by a misleading nearby lead. The engine reconstructs the path and returns expanded nodes, link count, summed review effort, runtime, and a plain-language explanation.

5.2 Module 2: Synthetic Evidence Graph

This module stores the example users, devices, processes, servers, alerts, and assets as nodes. It stores directed relationships with labels, review-effort costs, and synthetic confidence metadata as edges. It supports scenario selection and exposes selectable start and target nodes. The data is local and synthetic so that the demonstration is reproducible. No production security information is stored or requested.

5.3 Module 3: Graph Visualization and Comparison

This module draws the directed evidence graph in the browser. It distinguishes node types, highlights the source and target, and animates nodes expanded by the selected algorithm. It highlights the resulting investigation path after the animation completes. A comparison table presents link count, review effort, expanded nodes, runtime, and guarantees for all algorithms. Node details and edge tooltips help users understand the illustrative relationships and costs. The interface labels confidence and effort as synthetic contextual information.

5.4 Module 4: Flask API and Input Validation

This module serves the dashboard and provides JSON endpoints for scenarios, algorithm descriptions, search execution, and health checking. It validates algorithm names and ensures selected source/target nodes belong to the chosen graph. Invalid requests receive a clear client error. The API separates the Python search engine from the browser interface. It runs locally and does not require an external service or database.

6. System Limitations/Constraints

The graph, relationships, confidence values, and review costs are synthetic and simplified. A low-cost evidence path is not necessarily a real attack path or the most probable explanation. Greedy and Hill Climbing are not guaranteed to be optimal; Hill Climbing may get stuck. The prototype has no live log ingestion, authentication, production persistence, or autonomous response capability.

7. Tools and Technologies

Table 1. Tools and technologies for the project

Tool / technology	Version	Rationale
Python	3.10+	Implements graph search, path costs, and API logic
Flask	3.1.1	Lightweight local web application and JSON endpoints
Python heapq	Standard library	Priority queue for UCS, Greedy Best-First, and A*

Tool / technology	Version	Rationale
Python `collections.deque`	Standard library	FIFO queue for BFS and reverse-hop heuristic calculation
HTML5	Living Standard	Structure for the dashboard
CSS3	Modern browser support	Responsive security-operations interface and graph styling
JavaScript	ES2020+	Graph rendering, API requests, animation, and comparison view
JSON	RFC 8259	Local synthetic scenario data and API messages
Visual Studio Code	Any current release	Suggested editor and debugger; optional
Modern web browser	Current version	Runs the local dashboard

The implemented AI methods are uninformed search, informed heuristic search, and local search. No machine-learning or deep-learning model is included because it is outside this midterm project's implemented scope.

8. Conclusion

CyberTrail applies CSC262 search concepts to the cybersecurity task of exploring relationships among synthetic alert evidence. It formally represents the source, states, transitions, target, and path cost. Six algorithms make it possible to compare uninformed, informed, and local search on the same scenario. The graph animation and metrics make the algorithms' behavior easier to inspect and explain. The prototype is for education and analyst decision support only, not for real-world threat detection or automated incident response.

9. References

1. Russell, S. J., & Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.
2. CSC262: Artificial Intelligence-I lecture slides supplied for the midterm: intelligent agents and environments, search problem formulation, uninformed/informed search, heuristic functions, and local search.
3. Flask Documentation. https://flask.palletsprojects.com/
4. Python Documentation — `heapq`. https://docs.python.org/3/library/heapq.html